Sprint 4

Task & Workflow Overview

Team	Members	User Stories
Team 1	3 (2 BE + 1 FE)	US2 — Online Payment & Cash Shortfall Tracking
Team 2	2 (1 BE + 1 FE)	US4 — SA Data Price Suggestion | US5 — Trade Facility Utilisation | US6 — Analytics Export & Moderation Report | US3 Backend
Team 3	1 (BE + FE)	US1 — Seller Transaction History | US3 Frontend | Polish

Team 1 — Online Payment & Cash Shortfall Tracking

User Story: As a student, I want to pay the full or partial amount online, so that the payment is recorded before I collect the item.

Team Overview

Dev	Role	Scope
Dev A	Backend	Builds the payment model, shortfall model, and all payment API routes
Dev B	Backend	Modifies the transaction route to support payment statuses, adds shortfall checks before staff completion, and attaches shortfall data to staff queries
Dev C	Frontend	Builds the payment page, adds Pay Now buttons and payment badges to the student transactions view, and adds shortfall visibility + settle controls to the staff view

Yoco Integration — Setup & Responsibilities

Who sets up the Yoco account

- Dev A creates the Yoco account and obtains both API keys — the secret key (sk_test_...) and the public key (pk_test_...).

- Dev A adds the secret key to .env as YOCO_SECRET_KEY, then shares the public key with Dev C.

Dev A — Backend responsibilities

- Keeps the Yoco secret key in .env (never exposed to the client)
- In POST /payments/pay: reads yoco_token from the request body, sends a charge to Yoco's API using the secret key and amount, handles the response (success → proceed, failure → return 400)
- Tests the charge endpoint in Yoco test mode before going live

Dev C — Frontend responsibilities

- Includes Yoco's Inline JS SDK script in payment.html (loaded from Yoco's CDN)
- Initialises Yoco on the payment page using the public key
- When the user submits a card payment: SDK collects card details and returns a token
- Sends the token as yoco_token in the POST /payments/pay body
- Handles Yoco frontend errors (e.g. card declined, invalid card number)

Dev A and Dev C must agree on

- Request body shape: { transaction_id, amount_paid, payment_method: 'yoco', yoco_token }
- What the Yoco error response looks like so Dev C can display it correctly
- Testing in Yoco test mode before switching to live keys

Dev A — Backend Tasks

Task A1: Create models/payment.js

A Payment class with static methods wrapping Supabase operations on a new payments table.

Column	Type	Notes
id	uuid	primary key
transaction_id	uuid	foreign key to transactions
amount_paid	numeric(10,2)	
payment_method	text	yoco or cash
status	text	pending, paid, failed, refunded
paid_at	timestamptz	set when status becomes paid

Column	Type	Notes
created_at	timestamptz	default now

Methods:

- Payment.create({ transaction_id, amount_paid, payment_method, status }) — inserts a new row; defaults status to 'pending'; returns the created record
- Payment.findByTransaction(transaction_id) — returns all payments for a transaction ordered newest first
- Payment.updateStatus(id, status) — updates status; if new status is 'paid', also sets paid_at to now; returns updated record

Each method uses createSupabaseClient(). Handle errors by throwing so the route handler can catch them.

Task A2: Create models/shortfall.js

A Shortfall class with static methods wrapping operations on a new shortfalls table.

Column	Type	Notes
id	uuid	primary key
transaction_id	uuid	foreign key to transactions
amount_owed	numeric(10,2)	
status	text	outstanding, settled
created_at	timestamptz	default now
settled_at	timestamptz	nullable

Methods:

- Shortfall.create({ transaction_id, amount_owed }) — inserts with status 'outstanding'; returns created record
- Shortfall.findByTransaction(transaction_id) — returns all shortfalls ordered newest first (usually 0 or 1 per transaction)
- Shortfall.settle(id) — updates status to 'settled', sets settled_at to now; returns updated record

Task A3: Create routes/payments.js

An Express router with three endpoints. All use createSupabaseClient(req, res) for cookie-based auth.

POST /payments/pay — Buyer submits a payment

- Authenticate current user → 401 if not logged in

- Read transaction_id, amount_paid, payment_method (+ optional yoco_token) from body → 400 if required fields missing
- Look up transaction → 404 if not found
- Verify current user is the buyer_id → 403 if not
- Verify transaction status is 'in_progress' → 400 if not
- Look up listing to get full price; validate amount_paid is positive and does not exceed listing price → 400 if invalid
- If payment_method === 'yoco': send charge to Yoco API using token; return 400 with Yoco error if it fails
- If payment_method === 'cash': skip Yoco charge
- Create payment record via Payment.create() with status 'paid'
- If amount_paid < listing price: create shortfall record, update transaction to 'awaiting_payment'
- If amount_paid >= listing price: update transaction status to 'paid'
- Return { ok: true, payment: {...}, shortfall: {...} or null }

GET /payments/:transactionId — View payment status

- Authenticate → verify user is buyer or seller → 403 if neither
- Query all payments and shortfalls for this transaction
- Calculate total_paid, total_shortfall, fully_paid boolean (zeros if no records)
- Return { ok: true, payments: [...], shortfalls: [...], total_paid, total_shortfall, fully_paid }

PUT /payments/:transactionId/settle-shortfall — Staff confirms cash settlement

- Authenticate → check role is 'Staff' or 'Admin' → 403 if not
- Find outstanding shortfall → 404 if none
- Call Shortfall.settle(id); if no outstanding shortfalls remain, update transaction to 'paid'
- Return { ok: true, shortfall: {...} }

Register in server.js: app.use("/payments", paymentsRouter)

Dev B — Backend Tasks

Task B1: Modify routes/transactions.js — payment status support

Part 1 — Add comment at the top of the file documenting all valid statuses:

Status	Meaning
pending	Buyer initiated reserve/buy, no slot booked yet
in_progress	Slot booked, awaiting exchange
awaiting_payment	Partial payment made, shortfall outstanding
paid	Fully paid (no outstanding shortfall)
completed	Staff confirmed both drop-off and collection

Status	Meaning
cancelled	Transaction aborted

Part 2 — Add shortfall check before staff completion:

- In PUT /:id when action === 'complete': query shortfalls table for any row where transaction_id matches and status is 'outstanding'
- If outstanding shortfall exists → return 400: "Cannot complete transaction: outstanding shortfall of R[amount]. Please confirm settlement first."
- If no outstanding shortfall → proceed with existing completion logic

Task B2: Modify staff query to include shortfall info

- When fetching transactions for staff (in_progress and awaiting_payment statuses), also fetch all shortfall records in a separate query
- Build a lookup object: { [transaction_id]: shortfall_object }
- For each transaction in the response, attach a shortfall field:
  - No shortfall exists → shortfall: null
  - Shortfall outstanding → shortfall: { amount_owed, status: 'outstanding' }
  - Shortfall settled → shortfall: { amount_owed, status: 'settled' }

Dev C — Frontend Tasks

Task C1: Create pages/payment.html

- Standard student sidebar (consistent with student-transactions.html)
- Item Summary Card (read-only): item thumbnail, title, listed price
- Payment form: amount input (min 0.01, step 0.01), real-time hint text, payment method dropdown, Pay Now button, message area
- States: already fully paid → hide form; no transactionId in URL → show error; transaction not in_progress → explain why payment isn't available

Task C2: Create scripts/payment.js

- On page load: read transactionId from URL params; fetch GET /payments/{transactionId}; populate item summary; pre-fill amount with full price
- Real-time remaining calc: listen to input event; show "No shortfall — fully paid" or "Shortfall of R[x] will be recorded"
- Submit: validate amount and method → disable button → if yoco: collect token via Yoco SDK → POST /payments/pay
- On success: show message, redirect to student-transactions.html after 1.5 seconds
- On failure: show server error, re-enable button; handle network errors gracefully

Task C3: Modify scripts/student-transactions.js — payment buttons and badges

- in_progress: add "Pay Now" button → payment.html?transactionId={id}
- awaiting_payment: show yellow/orange badge "⚠ Shortfall"
- paid: show green badge "✓ Paid"
- Add click handler via event delegation for data-pay-transaction attribute
- Edge cases: missing listing data → show "Unknown item"; payment fetch failure → still render card

Task C4: Modify staff UI for shortfall visibility

- shortfall === null → grey badge "No shortfall" / "Paid in full"
- status === 'outstanding' → red badge "Shortfall: R[x] outstanding" + "Settle Shortfall" button
- status === 'settled' → green badge "Shortfall settled"
- Settle button: confirmation dialog → PUT /payments/{transactionId}/settle-shortfall → toast + refresh list; disable button during request

Team 2 — SA Data, Analytics & Moderation Backend

User Stories: US4 (SA Data Price Suggestion) | US5 (Trade Facility Utilisation) | US6 (Analytics Export & Moderation Report) | US3 Backend

Team Overview

Dev	Role	Scope
Dev D	Backend	SA data research + price suggestion service, all new analytics endpoints (slots, PDF, flagged content), moderation endpoints on ratings route, exclusion of removed reviews from public queries
Dev E	Frontend	Price suggestion on create-listing, slot utilisation chart on admin dashboard, PDF export button, flagged content stat cards, moderation page

Dev D — Backend Tasks

Task D1: Add moderation endpoints to routes/ratings.js

PUT /ratings/:id/flag — Any authenticated user flags a review

- Authenticate → 401 if not; look up rating → 404 if not found
- Set flagged = true and flagged_at = now(); return { ok: true, rating: {...} }
- No role restriction — any logged-in user can flag

PUT /ratings/:id/mark-safe — Admin clears a false flag

- Authenticate and verify 'Admin' role → 403 if not; look up rating → 404 if not found
- Set flagged = false and flagged_at = null; return { ok: true, rating: {...} }

PUT /ratings/:id/remove — Admin soft-deletes an abusive review

- Authenticate and verify 'Admin' role → 403 if not; look up rating → 404 if not found
- Set removed = true and removed_at = now(); return { ok: true, rating: {...} }
- Review is NOT deleted from the database — it is only hidden from public views

GET /ratings/moderation — Admin views all ratings with moderation status

- Authenticate and verify 'Admin' role → 403 if not
- Query ratings with joins: profiles (rater name), profiles (rated user name), transactions + listings (item title)
- Fields: rating ID, rater name, rated user name, item title, rating value, review text, flagged, flagged_at, removed, removed_at, created_at
- Order by created_at descending; return { ok: true, ratings: [...] }

Task D2: Modify routes/ratings.js — exclude removed from public GET /ratings

- When querying by user_id (ratings received): filter out removed = true from results and the average calculation
- When querying by rater_id (ratings given): do NOT filter — the original rater should see their own history
- Recommended approach: add .or('removed.is.null,removed.eq.false') to the Supabase query for public display

Task D3: Modify routes/listings.js — exclude removed from card rating averages

- In the enrichWithRatings function, filter the ratings query to exclude rows where removed = true
- Supabase: .is('removed', false) or .or('removed.is.null,removed.eq.false')
- Existing aggregation logic is unchanged — it just operates on a cleaner dataset

Task D4: Research SA data source → docs/backlog/sa-data-source.md

- Source name and URL (e.g. StatsSA Consumer Price Index — statssa.gov.za)
- How to access the data (CSV, REST API, manual copy-paste)
- Update frequency (monthly, quarterly, etc.)
- Category mapping table: Our Category → Source Category → Average Price
- How the suggestion is computed: source average × condition multiplier (New=100%, Good=85%, Fair=70%, Worn=50%, Damaged=30%)
- Limitations: which categories have no match; what happens when data is outdated

Task D5: Create services/price-suggestion.js

getSuggestionForCategory(category, condition = 'Good')

- Query price_suggestions table for matching category → return null if not found
- Apply condition multiplier: New=100%, Good=85%, Fair=70%, Worn=50%, Damaged=30%
- Return { category, base_price, adjusted_price, condition, condition_adjustment }

seedPriceSuggestions(supabase) — one-time use

Category	Base Price (from SA data research)
Textbooks	**R450**
Electronics	**R2 500**
Furniture	**R800**
Clothing	**R350**
Appliances	**R1 200**
Stationary	**R80**
Tickets	**R300**
Miscellaneous	**R200**

- Upsert into price_suggestions using category as conflict key — safe to run multiple times

Task D6: Add price suggestion endpoint to routes/listings.js

GET /listings/price-suggestion/:category?condition=Good

- Validate category is one of the 8 valid categories → 400 if not
- Call getSuggestionForCategory(category, condition)
- Return { ok: true, suggestion: {...} } or { ok: true, suggestion: null }
- No authentication required — public data lookup

Task D7: Add slot utilisation endpoint to routes/analytics.js

GET /analytics/slots?from=2026-01-01&to=2026-12-31

- Admin auth check via requireAdmin() → 403 if not admin
- Query trade_slots; filter by date range if from/to params provided
- Compute: total_slots (count), total_capacity (sum), total_booked (sum), utilisation_pct = (booked / capacity) x 100 rounded to 2dp; if capacity is 0 return utilisation 0
- Group by date for daily breakdown: { date, capacity, booked, utilisation_pct }
- Return { ok: true, summary: {...}, by_date: [...] }

Task D8: Add PDF export endpoint to routes/analytics.js

GET /analytics/export/pdf

- Admin auth check → 403 if not admin
- Fetch all report data in parallel: completed transactions, category data, slot utilisation, flagged content summary
- Generate PDF using pdfkit (npm install pdfkit) with these sections:
  - Title: "Campus Marketplace — Analytics Report" + generation date
  - Section 1: Summary — total completed transactions, total value (R), average per day
  - Section 2: Popular Categories — table with Category, Transaction Count, Total Value
  - Section 3: Transaction Trends — time period and transaction count
  - Section 4: Trade Facility Utilisation — total slots, capacity, booked, utilisation %
  - Section 5: Moderated Content — total reviews, flagged count, removed count
- Sections with no data show the header + "No data available"
- Set Content-Type: application/pdf and Content-Disposition: attachment; filename=analytics-report-{YYYY-MM-DD}.pdf

Task D9: Add flagged content endpoint to routes/analytics.js

GET /analytics/flagged-content

- Admin auth check → 403 if not admin
- Query ratings table for: total_reviews (all rows), flagged_count (flagged=true), removed_count (removed=true)
- Return { ok: true, summary: { total_reviews, flagged_count, removed_count } }
- Return zeros if table is empty — do not error

Dev E — Frontend Tasks

Task E1: Modify scripts/create-listing.js — show price suggestion

- After the category dropdown in the DOM, add a <p> element for suggestion text
- Listen to change events on both the category and condition dropdowns
- Fetch GET /listings/price-suggestion/{category}?condition={condition}
- Suggestion found → show "Suggested price: R[adjusted_price] (based on SA market data)"; optionally auto-fill price if empty
- Suggestion null → show "No price suggestion available for this category"
- Fetch failure → silently ignore; do not block the form
- Suggestion is purely advisory — no validation enforces it

Task E2: Modify pages/admin-dashboard.html — add slot utilisation section

- New stat card in the existing stats grid: heading "Slot Utilisation", value as utilisation %, sub-text "[booked] / [capacity] booked"

- New chart section below Category Distribution: title "Trade Facility Utilisation", bar chart canvas id="slotsChart"

Task E3: Modify scripts/analytics.js — add slot chart

- New function loadSlotUtilisation(): fetch GET /analytics/slots; update stat card; create Chart.js bar chart in #slotsChart
- Chart: dates on x-axis, Booked (#003b7e) and Capacity (#c9a84c) bars; y-axis starts at 0
- If by_date is empty: show "No slot data available" instead of an empty chart
- Call loadSlotUtilisation() in initPage() alongside existing load calls

Task E4: Add PDF export button to admin dashboard

- In pages/admin-dashboard.html: add "Export PDF" button (class: export-btn, id: export-pdf-btn) next to existing "Export CSV"
- In scripts/analytics.js: click handler → window.open('/analytics/export/pdf', '_blank')

Task E5: Add flagged content stat cards to admin dashboard

- Two new stat cards: "Flagged Reviews" (#flagged-count) and "Removed Reviews" (#removed-count)
- New function loadFlaggedContent(): fetch GET /analytics/flagged-content; set both count elements; on failure set both to "—"
- Call loadFlaggedContent() in initPage()

Team 3 — Seller History & Moderation Frontend

User Stories: US1 (Seller Transaction History) | US3 Frontend (Moderation page + flag button) | Polish

Team Overview

Dev	Role	Scope
Dev F	Full stack	Seller history route + page + card link, review flagging UI on seller history, admin moderation page and script, admin sidebar update, final polish pass

Dev F — Tasks

Task F1: Create routes/seller-history.js

GET /seller-history/:userId

- Look up user's profile in profiles table → 404 "User not found" if not found
- Query transactions where seller_id = userId AND status = 'completed'; join with listings (title, price, image) and profiles as buyer (buyer name)
- Query non-removed ratings where rated_id = userId; calculate average (rounded to 2dp) and total count

Response field	Shape
seller	{ id, name, member_since }
rating	{ average, total }
completed_transactions	[{ item_title, item_price, item_image, buyer_name, date }, ...]

- Edge cases: no transactions → empty array; no ratings → { average: 0, total: 0 }

Register in server.js: app.use("/seller-history", sellerHistoryRouter)

Task F2: Create pages/seller-history.html

- Standard student sidebar (same as dashboard.html)
- Seller Info Header: avatar placeholder, seller name (id="seller-name"), "Member since [date]" (id="seller-meta"), rating display (id="seller-rating")
- Reviews List: reviewer name, star rating, review text, date; Flag button if viewer is not the seller; empty state "No reviews yet."
- Completed Transactions: grid of cards (thumbnail, title, price, buyer name, date); empty state "No completed transactions yet."
- Reads sellerId from URL query parameter ?sellerId=xxx

Task F3: Create scripts/seller-history.js

- Verify viewer is logged in via auth.js; read sellerId from URL params → show "No seller specified" and stop if missing
- Fetch /seller-history/${sellerId}; populate header (name, member since formatted as "March 2026", rating); render transaction cards and review cards
- Flag button: PUT /ratings/{ratingId}/flag → show toast "Review flagged for moderator review", disable button; prevent double-flags
- Handle empty states and fetch errors gracefully

Task F4: Modify components/card.js — seller name becomes a clickable link

- Wrap seller name in an <a> tag pointing to /seller-history.html?sellerId=${listing.user_id}
- Add onclick="event.stopPropagation()" so clicking the name doesn't trigger parent card click

- Style the link (app theme link colour or underline); optionally append "View History" text
- If listing has no user_id: render name as plain text

Task F5: Polish Pass

- Verify all new routes are registered in server.js with app.use()
- Check all new HTML pages link to global.css and have correct sidebar with the right active state
- Test full US1 flow: card seller name → seller history page → transactions → reviews → flag button works
- Verify cross-team integration: completed transactions show correct payment status badges
- Confirm moderation page link exists in the admin sidebar
- Run npm test to ensure existing tests still pass
- Spot-check Sprint 3 features: reserve a listing, book a slot, complete a transaction, leave a rating
- Look for visual inconsistencies: same button classes, badge styles, and layout patterns as existing pages

Task F6: Add moderation link to admin dashboard sidebar

- In pages/admin-dashboard.html sidebar: add "Moderate Reviews" nav link pointing to moderation.html
- Use the same nav-item class to match existing style

Task F7: Create pages/moderation.html

- Standard admin sidebar with Moderate Reviews as the active item
- Filter buttons row: All | Flagged | Removed

Status	Badge Style	Action Buttons
Clean	Green / grey badge	Flag
Flagged	Orange badge	Mark Safe + Remove
Removed	Red badge	"Done" (greyed out, no action)

- Table columns: Reviewer Name, Reviewed User, Item, Rating, Review Text (truncated), Status badge, Actions
- Empty state: "No reviews found" when table is empty

Task F8: Create scripts/moderation.js

- Import requireRole from auth.js → redirect to login if not Admin
- Store all reviews in global array allReviews

- On page load: fetch /ratings/moderation → store response → call renderTable('all')
- renderTable(filter): all = all reviews; flagged = flagged=true && removed=false; removed = removed=true
- Filter button clicks: toggle active class on clicked button, call renderTable(filter)
- Event delegation on table body:
  - data-action="flag" → PUT /ratings/{id}/flag → re-render
  - data-action="mark-safe" → PUT /ratings/{id}/mark-safe → re-render
  - data-action="remove" → confirm dialog → PUT /ratings/{id}/remove → re-render
- On success: optional toast; on error: alert with server message

File Summary — All Teams

Team	New Files	Modified Files
Team 1	models/payment.js, models/shortfall.js, routes/payments.js, pages/payment.html, scripts/payment.js	routes/transactions.js, routes/slots.js, scripts/student-transactions.js, server.js, staff shortfall page
Team 2	services/price-suggestion.js, docs/backlog/sa-data-source.md	routes/analytics.js, routes/listings.js, routes/ratings.js, pages/admin-dashboard.html, scripts/analytics.js, scripts/create-listing.js
Team 3	routes/seller-history.js, pages/seller-history.html, scripts/seller-history.js, pages/moderation.html, scripts/moderation.js	components/card.js, server.js, pages/admin-dashboard.html

Recommended Build Order

Phase	What happens	Who
Phase 1	Parallel — Payment models + routes | SA data research + moderation endpoints | Seller-history route + page	Dev A, Dev D, Dev F
Phase 2	Parallel — Transaction flow modification + staff queries | Admin dashboard UI + price suggestion | Seller card link + moderation page	Dev B, Dev E, Dev F

Phase	What happens	Who
Phase 3	Parallel — Payment UI + student-transactions + staff shortfall UI | Price-suggestion endpoint + PDF export	Dev C, Dev D
Phase 4	Integration testing: verify cross-team flows end-to-end. Polish: visual consistency, error states, npm test	Everyone